

Report Challenge: Find Key and Open Door with DQN

Student Number: K51843919

Student Name: Lukas Steu

Setup

As the challenge suggests, the code was run in *Google Colab*. Prior to that, an attempt was made to get the code running in *VS Code*, but soon after the first start, it has been noticed that the module *termios* could not be found. This is because the mounting of Google Drive can only be done *in Google Colab* in an easy way. The idea of installing *Google Colab* locally and get the code going has arisen but was deemed as too tedious rather quickly. As for *Google Colab* itself, the runtime type has been chosen to be of type *Python 3*. Additionally, a *T4 GPU* has been selected as the hardware accelerator, as well as the runtime version *Latest (recommended)*. After running a couple of models, the free resources provided by *Google Colab* had been spent, while not achieving a sufficient score. Therefore, additional resources have been purchased (100 compute units), to which 97.76 remain.

Changes of the Code

First, the challenge provides a very generous template of the code with some parts being left out for completion. While some parts only had to be completed, other snippets and cells have been added to the code. The following paragraphs will not explain everything in too much detail as the code itself provides a better understanding of what is happening, but a short description of the changes done to the code will be described.

Initially, the *Setup Notebook (Virtual Machine)* has been altered as the install command for *onnxscript* has been added to properly export the *.onnx* file.

Furthermore, the cell containing the hyperparameters for the model has been modified.

- The *num_episodes* parameter has been increased from 1000 to 1500 to get more checkpoints resulting in a better performance.
- The *minibatch_size* has been increased from 5 to 64 as the initial value created too much noise.
- The *eval_episode* parameter has been decreased from 100 to 50 so the model will act faster on identifying good models.
- The *num_eval* parameter has been increased from 10 to 20 to decrease the variance.
- The *epsilon_lb* parameter, most importantly, has been decreased from 0.3 to 0.05 as this means that the model will not be capped at a random decision of 30%, which made it greedier. This had the greatest impact on the model's performance.

Furthermore, the *Load model (if available)* cell has been changed. The command for saving the checkpoints has been added so the checkpoints will be saved in *Google Drive*. Additionally, the second *save_path* command has been commented so the first command, which also defines the saving path to *Google Drive* is not overwritten.

The first lines of code to be completed were executed in the *soft_update* function. Here the weights of the target model are slightly mixed into the current model by a factor of *tau*. They are just copied from the target network and adjusted by the provided formula. With *tau* being 0.1, this means that 99.9% remains from the target network.

Next up was the implementation of the actual *DQN training loop* cell. Here the storing of the transition in the replay buffer has been realised, as well as the update of the current state and the timesteps. Additionally, the minibatches had to be sampled at random by a single command resulting from the buffer class. Logically, the computation of the q-values was up next. Here the states had to be transformed to a tensor and put through the dqn's of the checkpoint. At last, predictions were calculated by summing up the q-values multiplied by the actions, which were then processed to compute the loss (here MSE). All these modifications were meant to be done. Another modification which was not foreseen has been implemented in the last if-statement. Here, the checkpoint path is now updated and saved for each checkpoint instead of just moving to last one. Consequently, a cell has been added, sorting & printing all the available checkpoints. The evaluation of these checkpoints is also changed. Instead of just evaluating the last checkpoint as it has been proposed initially, every checkpoint is now evaluated. Afterwards, the model is selected and saved. A new cell is added to load the new path of saved model, which is then tested at last, resulting in an *Average Return* of 0.91035.

Runtime

Out of personal interest (and because someone mentioned it on the discord channel), the runtimes for cells with a longer runtime have been tracked. With the setup described above, the runtime for executing the installs was 1min 24s. The training loop took 32min 24s to complete and the evaluation of all checkpoints has been accomplished within 4min 11s.